

ML Assignment 3

Name: Dhakshin K

Entry Number: 2024AIB1009

1 Q1 Observations

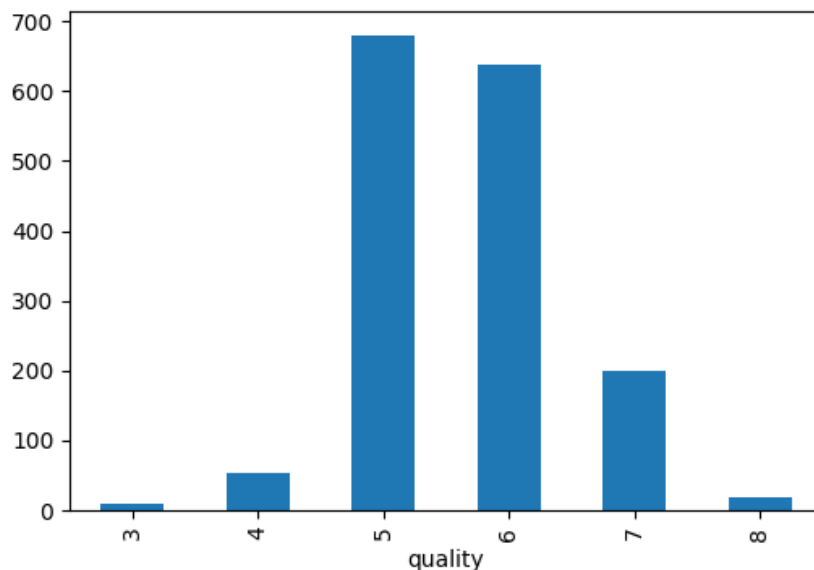


Figure 1: Distribution of Quality

From the image, most of the samples are concentrated around quality 5 and 6. These two categories have the highest counts compared to others.

Quality 5 has slightly more samples than quality 6, so it looks like average quality is most common in the dataset.

Very low quality values like 3 and 4 have very few samples. Similarly, very high quality like 8 also has very less data.

Quality 7 has moderate number of samples but still much lower than 5 and 6.

Overall, the data is not evenly distributed and is heavily centered around mid quality values.

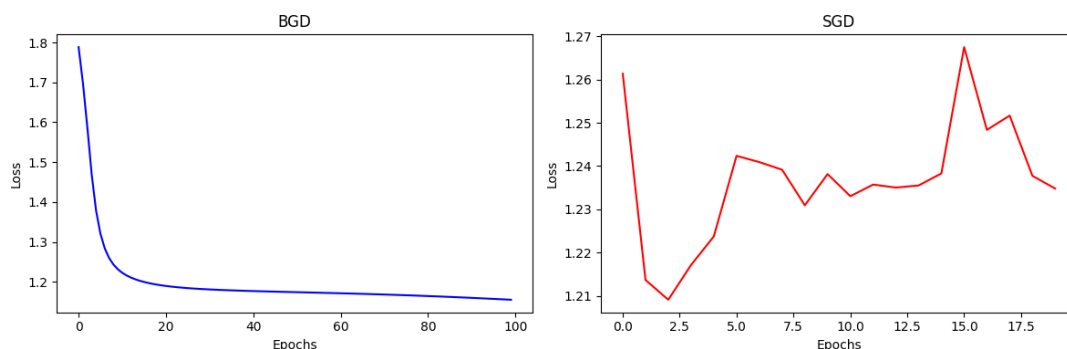


Figure 2: Loss vs Epochs for BGD and SGD

From the left graph (BGD), the loss decreases smoothly as epochs increase. It shows a stable convergence and does not fluctuate much.

In the beginning, the loss drops quickly and then it decreases slowly. This means the model is learning fast at first and then gradually improving.

From the right graph (SGD), the loss is not smooth and keeps fluctuating. It goes up and down at different points.

Even though SGD reaches lower loss quickly, it is not stable compared to BGD. Overall, BGD is more stable while SGD is faster but noisy during training.

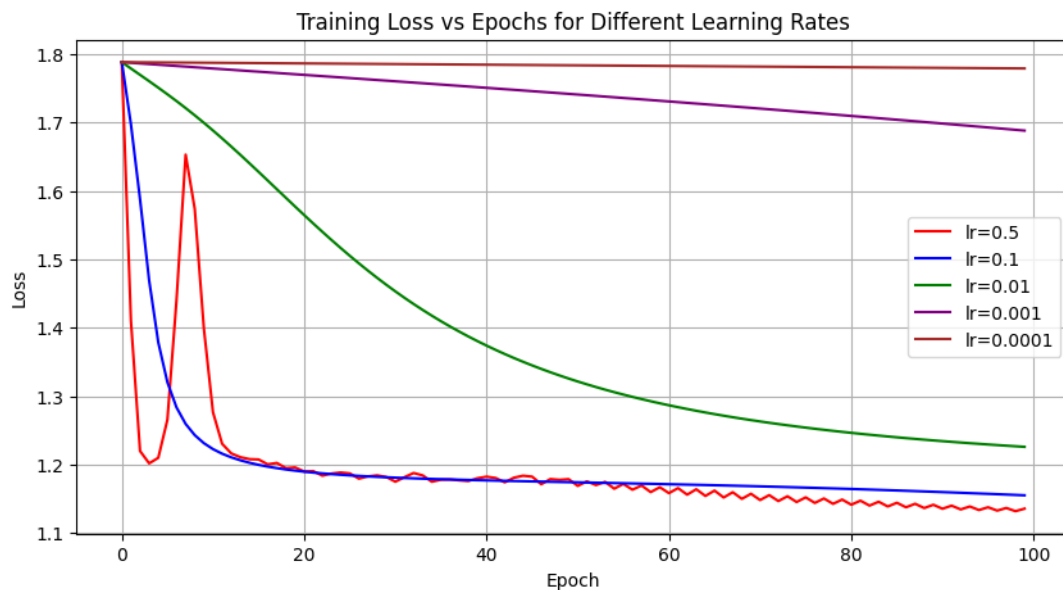


Figure 3: Training Loss vs Epochs for Different Learning Rates

From the graph, different learning rates affect how fast and stable the model learns.

For $lr = 0.5$, the loss fluctuates a lot in the beginning and is not stable. It shows that too high learning rate causes instability.

For $lr = 0.1$, the loss decreases quickly and then becomes stable. This seems like a good learning rate.

For $lr = 0.01$, the loss decreases smoothly but slowly compared to $lr = 0.1$.

For very small learning rates like 0.001 and 0.0001, the loss decreases very slowly and almost looks flat. Learning is very slow in these cases.

Overall, a moderate learning rate like 0.1 gives better and faster convergence compared to very high or very low values.

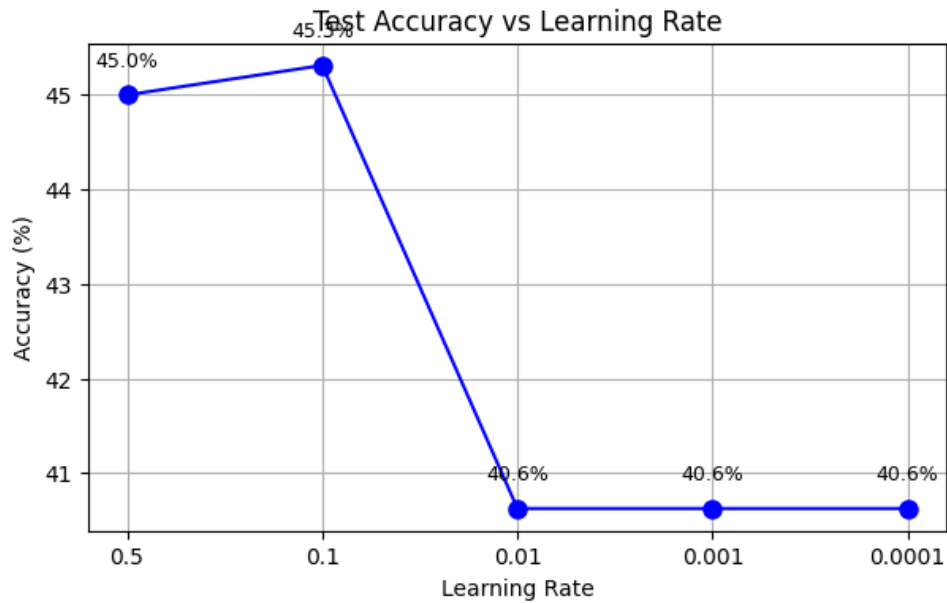


Figure 4: Test Accuracy vs Learning Rate

From the graph, the best accuracy is achieved at learning rate 0.1 which is around 45.3%.

For learning rate 0.5, the accuracy is slightly lower at around 45%. This may be due to instability at higher learning rate.

For smaller learning rates like 0.01, 0.001 and 0.0001, the accuracy drops to around 40.6%.

This shows that very small learning rates are not able to learn properly and give poor performance.

Overall, learning rate 0.1 gives the best result among all values.

2 Q2 Observations

In this question, a classification task is performed on hand gesture images representing digits from 0 to 9.

The dataset consists of images of hands showing different finger combinations. Each image belongs to a specific class label from 0 to 9.

Before training the model, the data is preprocessed and organized properly so that it can be used for learning.

A model is then trained to classify these images into correct digit classes based on the hand gesture.

The performance of the model depends on how well it can distinguish between similar looking gestures and generalize on unseen data.

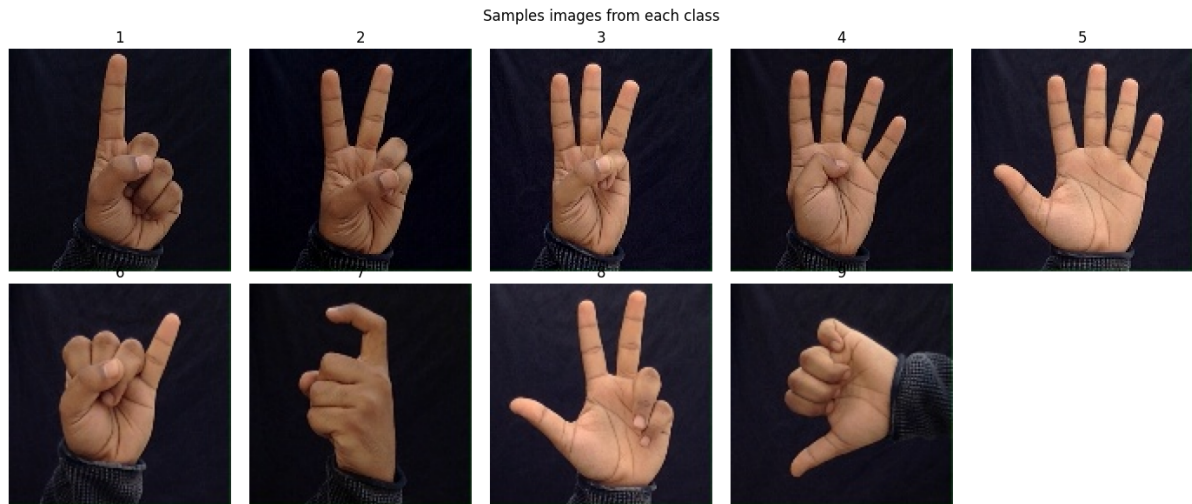


Figure 5: Sample Images from Each Class (0-9)

From the image, it shows sample hand gestures for different classes from 0 to 9.

Each class represents a number using fingers. The images are clear and background is mostly dark, which helps in identifying the hand easily.

The variation in hand shapes is visible across different classes. Some classes like 1, 2, 5 are easy to distinguish, while others may look slightly similar.

There is also slight variation in hand orientation and finger positions, which can make classification a bit difficult.

Overall, the dataset looks well structured and suitable for training a classification model.

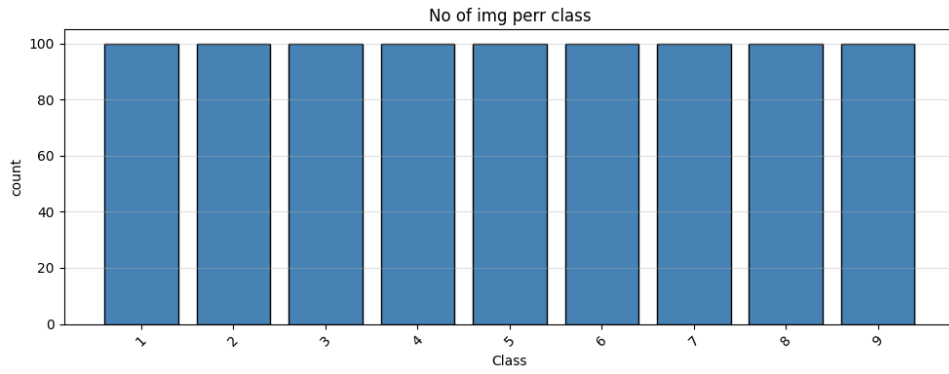


Figure 6: Number of Images per Class

From the graph, all classes have equal number of images.

Each class from 0 to 9 contains around the same count, which means the dataset is balanced.

Since the data is evenly distributed, the model will not be biased towards any particular class.

This is good for training because each class will be learned equally.

Overall, the dataset distribution looks proper and well balanced for classification.

Canny Edge Detection results

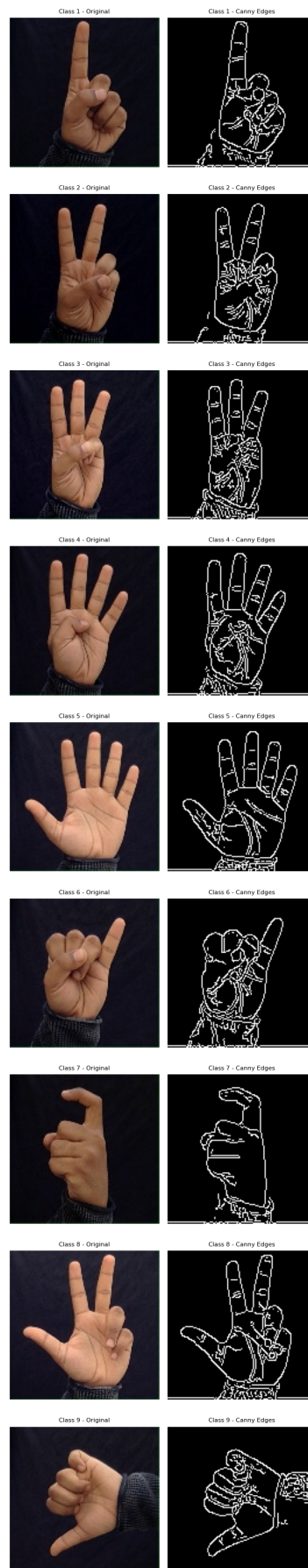


Figure 7: Canny Edge Detection Results for Different Classes

From the images, Canny edge detection is applied on each hand gesture image.

The edges of the fingers and hand shape are clearly visible in the processed images. Background is mostly removed and only important boundaries are highlighted.

This makes the shape of the hand more clear and easier to identify compared to original images.

However, some extra edges and noise can also be seen in few images, especially around palm lines.

Overall, edge detection helps in extracting important features which can improve model performance.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 128, 128, 32)	896
conv2d_1 (Conv2D)	(None, 128, 128, 32)	9,248
batch_normalization (BatchNormalization)	(None, 128, 128, 32)	128
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_2 (Conv2D)	(None, 32, 32, 32)	9,248
conv2d_3 (Conv2D)	(None, 32, 32, 24)	6,936
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 24)	96
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 24)	0
conv2d_4 (Conv2D)	(None, 8, 8, 24)	5,208
conv2d_5 (Conv2D)	(None, 8, 8, 16)	3,472
batch_normalization_2 (BatchNormalization)	(None, 8, 8, 16)	64
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 16)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 16)	0
dense (Dense)	(None, 128)	2,176
dense_1 (Dense)	(None, 128)	16,512
dropout (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 9)	1,161

Total params: 55,145 (215.41 KB)
Trainable params: 55,001 (214.85 KB)
Non-trainable params: 144 (576.00 B)

Figure 8: CNN Model Architecture Summary

From the image, the model used is a Sequential CNN model with multiple convolution layers.

The model starts with Conv2D layers which extract features from input images. These layers help in detecting patterns like edges and shapes.

Batch normalization layers are used to stabilize and speed up the training process.

Max pooling layers are applied to reduce the spatial size of feature maps and reduce computation.

As the network goes deeper, the feature map size decreases while important features are retained.

Global average pooling is used before fully connected layers to reduce dimensions.

Dense layers are used at the end for classification, and dropout is applied to prevent overfitting.

The final layer has 9 output units which represent the different classes.

Overall, the model is well structured for image classification tasks.

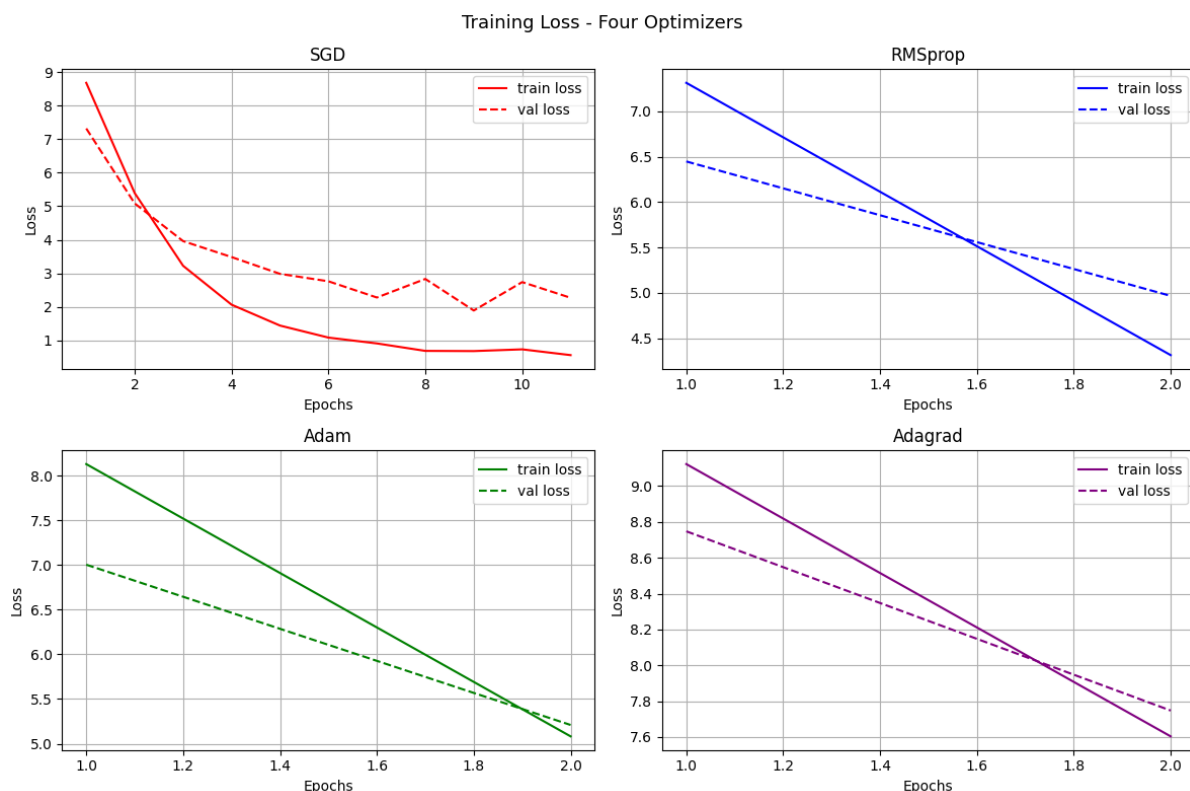


Figure 9: Training and Validation Loss for Different Optimizers

From the graphs, different optimizers show different learning behavior.

In SGD, the training loss decreases steadily but validation loss fluctuates. This shows that SGD is learning but not very stable.

RMSprop shows smooth decrease in both training and validation loss. It is more stable compared to SGD.

Adam optimizer also shows consistent decrease and performs better than SGD. It converges faster and more smoothly.

Adagrad shows slower decrease in loss compared to others. It is learning but not very efficiently.

Overall, Adam and RMSprop perform better in terms of stability and convergence, while SGD is slower and more noisy.

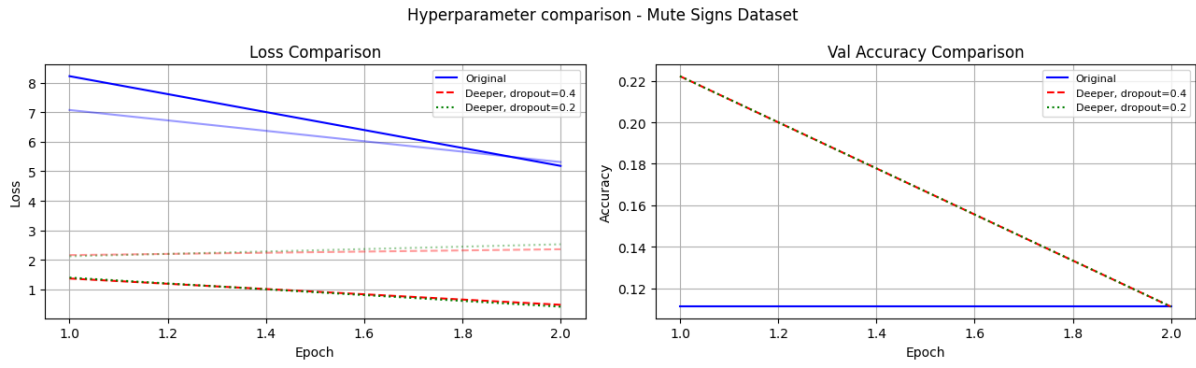


Figure 10: Hyperparameter Comparison with Different Model Configurations

From the graphs, different model configurations are compared using loss and validation accuracy.

In the loss graph, the deeper models show lower loss compared to the original model. This indicates better learning.

The model with dropout 0.2 performs slightly better than dropout 0.4, showing more stable decrease in loss.

In the validation accuracy graph, all models show similar trend and accuracy decreases over epochs.

The original model has very low accuracy compared to deeper models.

Overall, deeper models perform better in terms of loss, but accuracy improvement is not very significant.

3 Q3 Observations

In this question, a Recurrent Neural Network (RNN) model is used for training.

The goal is to understand how the model learns over iterations and how the training loss behaves.

RNNs are generally used for sequential data, where the model learns patterns over time steps.

The training process is observed by plotting loss with respect to iterations.



Figure 11: RNN Training Loss vs Iterations

From the graph, the training loss is increasing as the number of iterations increases. Initially, the loss starts at a lower value but keeps rising steadily over time.

This indicates that the model is not learning properly and is diverging instead of converging.

The increase in loss may be due to issues like high learning rate, poor initialization, or instability in training.

Overall, the RNN model is not performing well in this case as loss should ideally decrease during training.

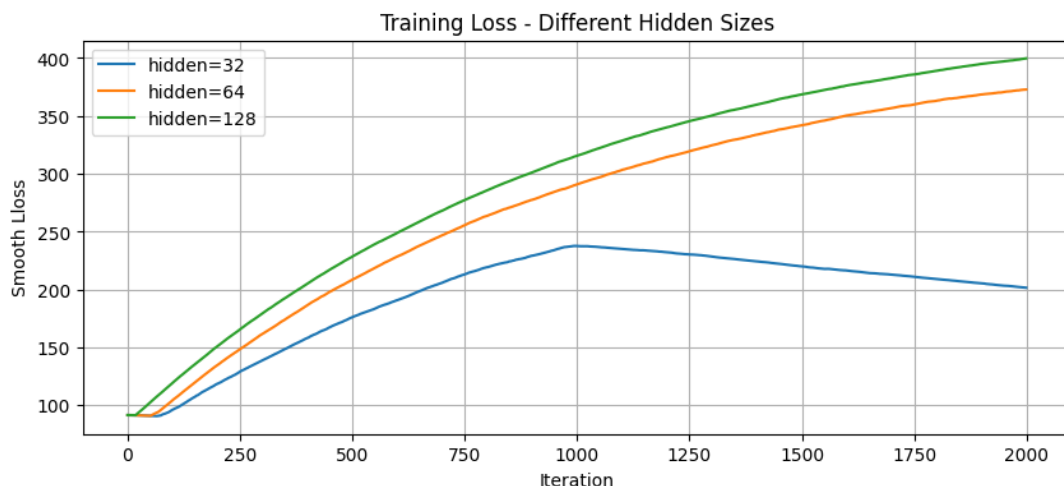


Figure 12: Training Loss for Different Hidden Sizes

From the graph, different hidden sizes affect the training loss behavior of the RNN.

For hidden size 32, the loss initially increases but then starts to decrease after some iterations. It shows relatively better stability compared to others.

For hidden sizes 64 and 128, the loss keeps increasing continuously as iterations increase.

Larger hidden sizes lead to higher loss, which indicates that the model is not able to train properly with increased complexity.

This may be due to issues like overfitting, vanishing or exploding gradients, or unstable training.

Overall, smaller hidden size performs better in this case, while larger hidden sizes make the training unstable.

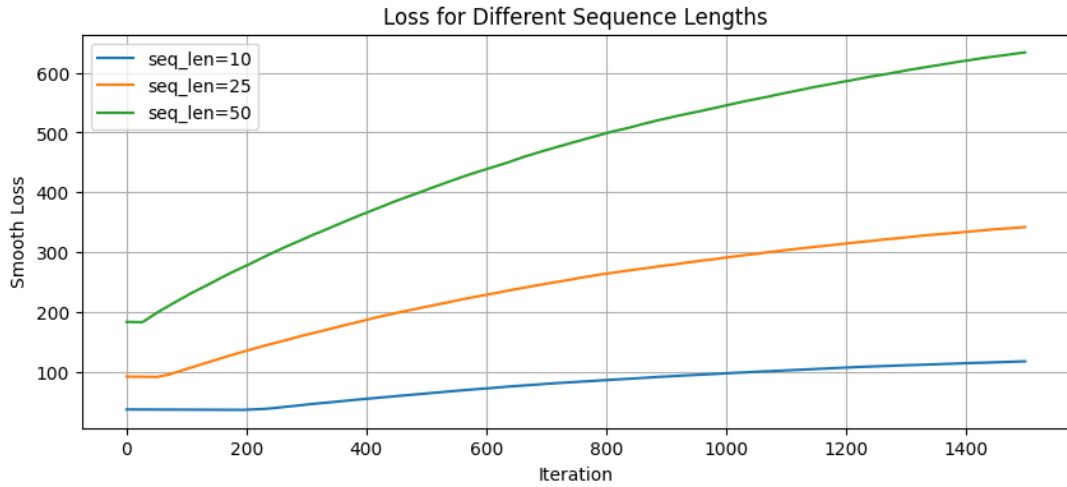


Figure 13: Training Loss for Different Sequence Lengths

From the graph, sequence length has a strong effect on training loss.

For sequence length 10, the loss increases slowly and remains relatively low compared to others.

For sequence length 25, the loss increases more compared to sequence length 10, showing reduced stability.

For sequence length 50, the loss increases rapidly and becomes very high as iterations increase.

This shows that larger sequence lengths make training more difficult for the RNN.

The model struggles to learn long dependencies, which leads to higher loss and unstable training.

Overall, smaller sequence length performs better, while larger sequence lengths cause poor performance.